



中国海洋大学  
OCEAN UNIVERSITY OF CHINA

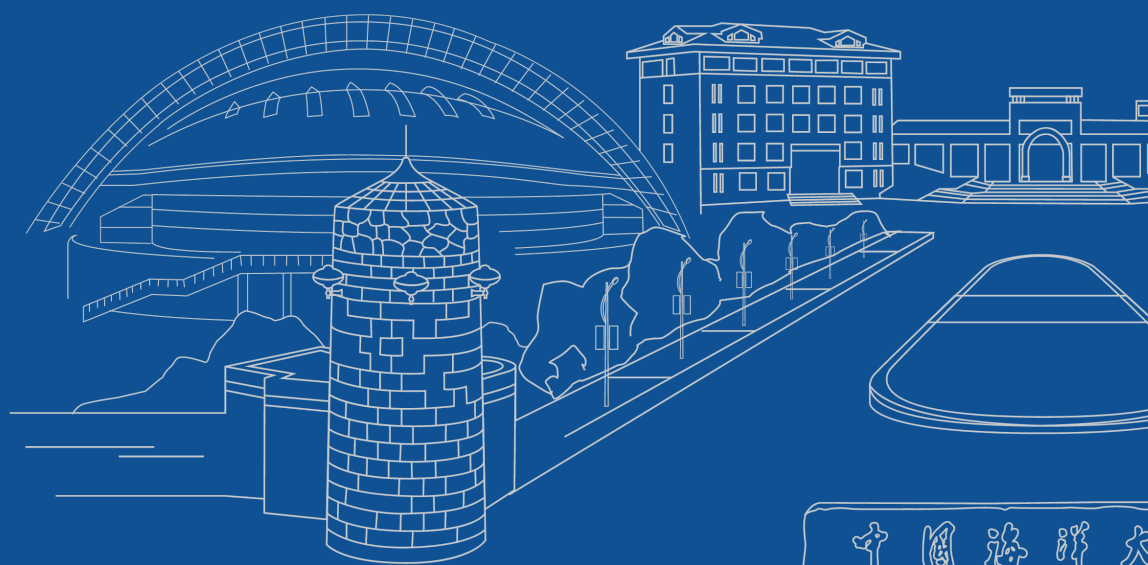
# 第二周实验报告

系统开发工具基础

刘晨希

25020007076

信息科学与工程学部  
计算机科学与技术学院  
计算机科学与技术专业  
系统开发工具基础



中国海洋大学

2026 年 9 月 6 日



中国海洋大学  
OCEAN UNIVERSITY OF CHINA

# 第二周实验报告

系统开发工具基础

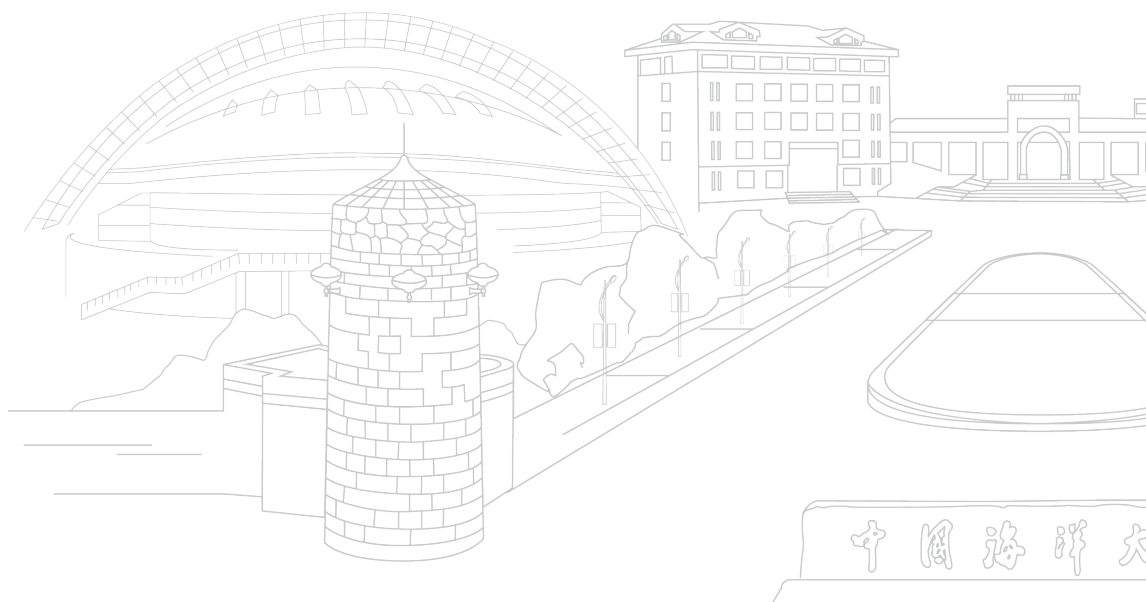
刘晨希

25020007076

指导老师: 周小伟

信息科学与工程学部  
计算机科学与技术学院  
计算机科学与技术专业  
系统开发工具基础

2026 年 9 月 6 日



# 目录

|                                 |           |
|---------------------------------|-----------|
| 目录                              | i         |
| <b>1 课堂练习及结果</b>                | <b>2</b>  |
| 1.1 第 5 题控制一个可清理的后台任务           | 2         |
| 1.1.1 实验内容                      | 2         |
| 1.1.2 实验结果                      | 3         |
| 1.2 第 6 题语义重构与本地开发反馈            | 4         |
| 1.2.1 实验内容                      | 4         |
| 1.2.2 实验结果                      | 7         |
| 1.3 第 7 题用调试器定位归并排序缺陷           | 7         |
| 1.3.1 实验内容                      | 7         |
| 1.3.2 实验结果                      | 9         |
| 1.4 第 8 题先测量，再优化慢速词频程序          | 9         |
| 1.4.1 实验内容                      | 9         |
| 1.4.2 实验结果                      | 11        |
| 1.5 Github 仓库地址                 | 11        |
| <b>2 课后练习内容和结果</b>              | <b>12</b> |
| 2.1 基础命令与重定向                    | 12        |
| 题 1: 确认 Shell 环境                | 12        |
| 题 2: ls -l 的输出格式                | 12        |
| 题 3: glob 模式匹配                  | 13        |
| 题 4: 三种引号的区别                    | 14        |
| 题 5: 标准流与重定向                    | 14        |
| 题 6: 退出状态与 &&/                  | 15        |
| 2.2 脚本编写与权限                     | 16        |
| 题 7: 为什么 cd 必须是 Shell 内建命令?     | 16        |
| 题 8: 写脚本检查文件是否存在                | 16        |
| 题 9: chmod +x 的作用               | 17        |
| 题 10: set -x 的作用                | 18        |
| 题 11: 带日期的备份文件                  | 19        |
| 题 12: 修改 flaky test 脚本接收命令行参数   | 19        |
| 2.3 管道、工具与进阶                    | 21        |
| 题 13: 找出 home 目录最常见的 5 种文件扩展名   | 21        |
| 题 14: xargs 结合 find 统计 .sh 文件行数 | 21        |

---

|                                            |           |
|--------------------------------------------|-----------|
| 题 15: curl 获取课程网站 HTML, 统计列出了多少讲 . . . . . | 22        |
| 题 16: jq 处理 JSON . . . . .                 | 23        |
| 题 17: awk 按列过滤并交换列 . . . . .               | 24        |
| 题 18: 拆解 SSH 日志管道 + 从历史记录找出最常用命令 . . . . . | 25        |
| <b>3 解题感悟</b>                              | <b>27</b> |

## 课堂练习及结果

### 1.1 第 5 题控制一个可清理的后台任务

#### 1.1.1 实验内容

如图 1.1 所示,使用 vim 编辑器创建 worker.sh。脚本通过 trap 命令设置了 TERM 和 INT 两个信号的处理函数,当脚本收到这两个信号时,会执行 echo CLEAN\_EXIT >> cleanup.log; exit 0,即向 cleanup.log 写入 CLEAN\_EXIT 后以退出码 0 正常结束;随后进入无限循环,每秒钟打印一次计数器 n 并将其自增。

```
liuchenxi@liuchenxideMacBook-Air q05 % vim worker.sh
```

图 1.1: 创建 worker.sh

如图 1.2 所示,使用 chmod +x 命令为 worker.sh 添加执行权限。

```
liuchenxi@liuchenxideMacBook-Air q05 % chmod +x worker.sh
```

图 1.2: 为 worker.sh 添加执行权限

如图 1.3 所示,在前台启动 worker.sh,并通过重定向把 stdout 写入 stdout.log、stderr 写入 stderr.log;随后使用 Ctrl-Z 挂起该任务,从图中可以看到 zsh 输出了 suspended 提示,说明任务已被挂起。

```
liuchenxi@liuchenxideMacBook-Air q05 % ./worker.sh > stdout.log 2> stderr.log
^Z
zsh: suspended ./worker.sh > stdout.log 2> stderr.log
```

图 1.3: 前台启动 worker.sh 并挂起

如图 1.4 所示,使用 bg 命令让被挂起的任务转到后台继续运行,并使用 jobs 命令确认任务状态。从图中可以看到,jobs 输出 [1] + running ./worker.sh > stdout.log 2> stderr.log,说明任务正在后台运行。

```
liuchenxi@liuchenxideMacBook-Air q05 % bg
[1] + continued ./worker.sh > stdout.log 2> stderr.log
liuchenxi@liuchenxideMacBook-Air q05 % jobs
[1] + running ./worker.sh > stdout.log 2> stderr.log
```

图 1.4: 使用 bg 让任务后台继续运行

如图 1.5 所示, 使用 `jobs -p` 取得后台任务的进程号 (PID), 并通过 `kill` 命令发送 `SIGTERM` 信号结束任务, 全程没有手工抄写 PID, 也没有使用 `kill -9`。任务收到 `SIGTERM` 信号后触发了 `trap` 中设置的清理逻辑, 最终以 `[1] + done` 结束 (`done` 表示进程以退出码 0 正常退出, 而非被信号强制终止), 随后 `jobs` 命令输出为空, 说明任务已经退出。

```
liuchenxi@liuchenxideMacBook-Air q05 % kill -TERM $(jobs -p)
kill: illegal pid: [1]
kill: illegal pid: +
kill: illegal pid: running
kill: illegal pid: ./worker.sh
kill: illegal pid: >
kill: illegal pid: stdout.log
kill: illegal pid: 2>
kill: illegal pid: stderr.log
liuchenxi@liuchenxideMacBook-Air q05 %
[1] + done      ./worker.sh > stdout.log 2> stderr.log
liuchenxi@liuchenxideMacBook-Air q05 % jobs
```

图 1.5: 发送 `SIGTERM` 信号结束任务

在图 1.5 中, `kill` 为发送信号命令, `-TERM` 表示发送 `SIGTERM` (终止) 信号, `jobs -p` 用于输出后台任务的进程号。需要注意的是, `zsh` 中直接执行 `kill -TERM $(jobs -p)` 时, 命令替换 `$(jobs -p)` 展开后得到的是任务的完整状态行而非纯 PID, 导致出现 `illegal pid` 报错, 需要先正确提取出纯 PID 再发送信号。

如图 1.6 所示, 使用 `grep` 命令在 `cleanup.log` 中查找 `CLEAN_EXIT`, 输出结果为 `CLEAN_EXIT`, 说明脚本的 `trap` 清理逻辑被正确触发, 任务已经干净地退出。

```
liuchenxi@liuchenxideMacBook-Air q05 % grep "CLEAN_EXIT" cleanup.log
CLEAN_EXIT
```

图 1.6: 确认 `cleanup.log` 中出现 `CLEAN_EXIT`

## 1.1.2 实验结果

第五题的 github 的 commit 记录和 push 记录如图 1.7 所示。

```
liuchenxi@liuchenxideMacBook-Air w2 % git add q05
liuchenxi@liuchenxideMacBook-Air w2 % git commit -m "完成了q05"
[main 9f36f10] 完成了q05
 4 files changed, 110 insertions(+)
 create mode 100644 w2/q05/cleanup.log
 create mode 100644 w2/q05/stderr.log
 create mode 100644 w2/q05/stdout.log
 create mode 100755 w2/q05/worker.sh
liuchenxi@liuchenxideMacBook-Air w2 % git push
[Enter passphrase for key '/Users/liuchenxi/.ssh/id_ed25519':
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 10 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (8/8), 764 bytes | 764.00 KiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:lcx821/lcx_course_work.git
   b92e9c1..9f36f10  main -> main
```

图 1.7: github 记录 Q5

## 1.2 第 6 题语义重构与本地开发反馈

### 1.2.1 实验内容

按照题目给定内容,在 q06 目录中创建 `math_utils.py`、`app.py` 和 `test_math_utils.py` 三个 Python 文件,并在 VSCode 中打开 q06,启用 Pylance (Python 语言服务器)。其中 `math_utils.py` 定义了函数 `total_price`, `app.py` 和 `test_math_utils.py` 分别导入并调用、测试该函数。

如图 1.8 和图 1.9 所示,在 `app.py` 中把光标放到 `total_price` 上,Pylance 悬浮显示该函数的定义信息(参数类型和返回类型);再通过“跳转到定义”(Go to Definition)定位到 `math_utils.py` 中函数 `total_price` 的定义处。

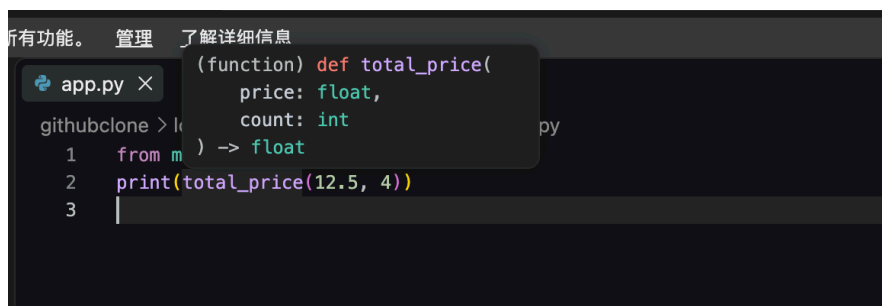


图 1.8: 悬浮显示 `total_price` 定义

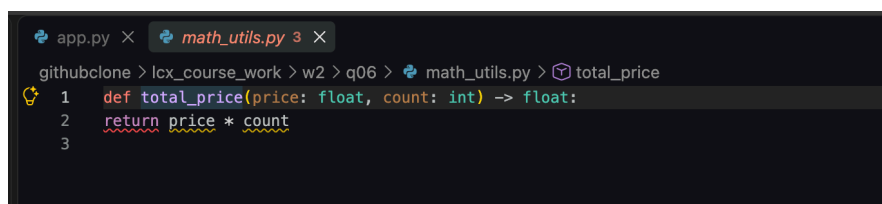


图 1.9: 跳转到 `math_utils.py` 中的定义

如图 1.10 所示,使用“查找所有引用”(Find All References)功能,可以看到 `total_price` 在 `app.py`、`math_utils.py` 和 `test_math_utils.py` 中的全部引用位置。

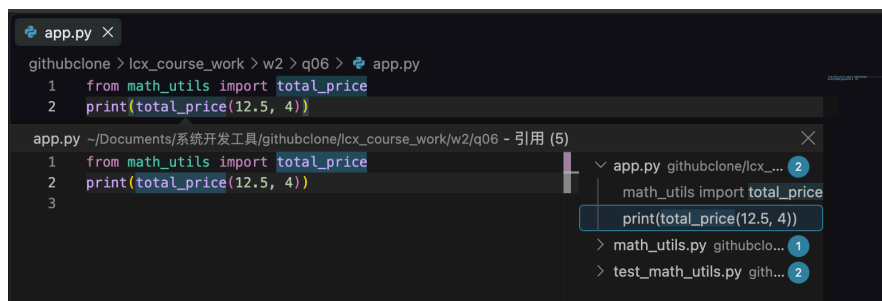


图 1.10: 查找 `total_price` 的所有引用

如图 1.11 和图 1.12 所示,使用“重命名符号”(Rename Symbol)把 `total_price` 重命名为 `calculate_total`,语言服务器会同步修改函数定义以及所有代码引用,保证定义与引用一致。

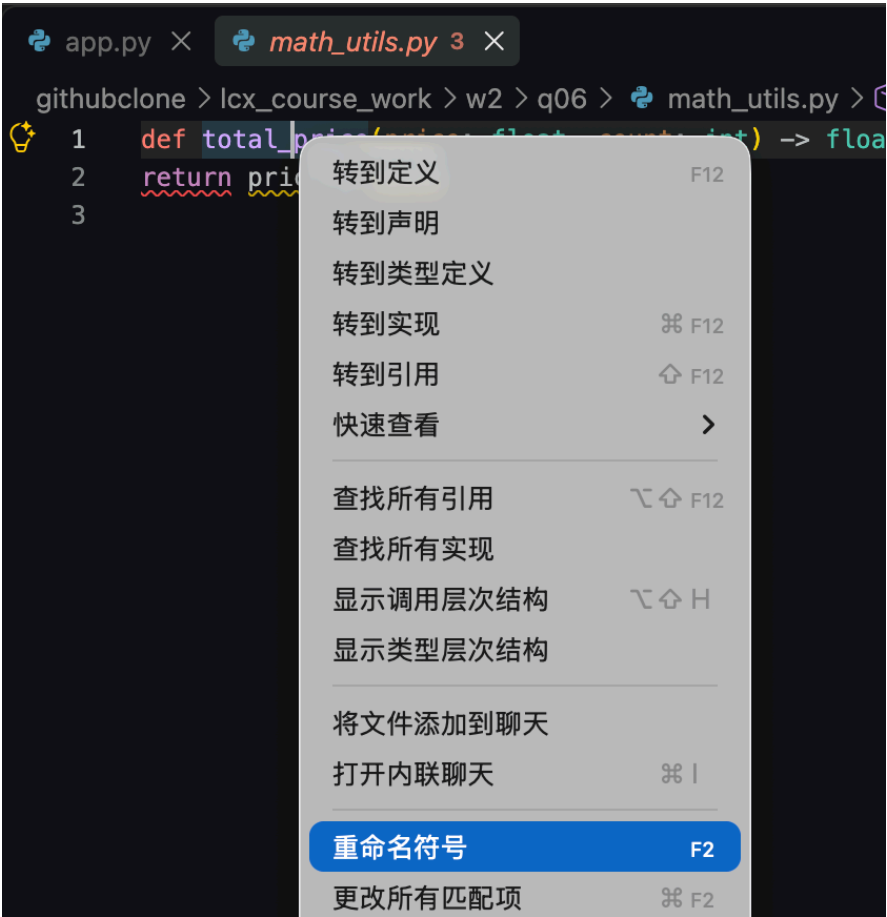


图 1.11: 使用重命名符号重命名 `total_price`

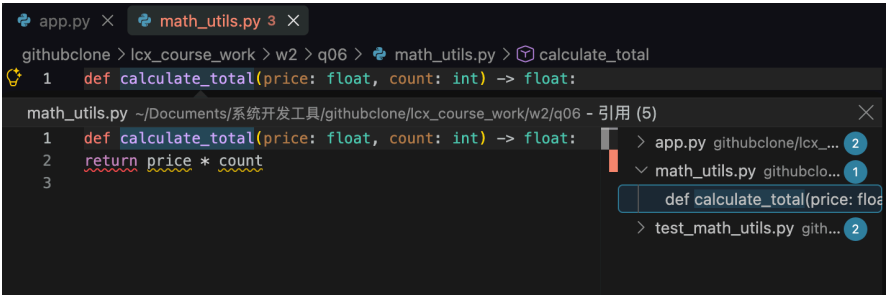


图 1.12: 重命名完成后的 `calculate_total`

如图 1.13 所示，在 `app.py` 中临时加入未使用的 `import os`，用于后续使用 `ruff` 检测并删除。



```

githubclone > lcx_course_work > w2 > q06 > app.py
1  import os
2
3  from math_utils import calculate_total
4  print(calculate_total(12.5, 4))
5

```

图 1.13: 临时加入未使用的 `import os`

如图 1.14 所示，运行 `ruff check app.py`，`ruff` 检查出两个问题：F401 表示 `os` 被导入但未使用，I001 表示 `import` 块未排序或未格式化。

```

liuchenxi@liuchenxideMacBook-Air q06 % ruff check app.py
I001 [*] Import block is un-sorted or un-formatted
--> app.py:1:1

1 | / import os
2 | |
3 | | from math_utils import calculate_total
4 | | print(calculate_total(12.5, 4))
  | ^
help: Organize imports
3 | from math_utils import calculate_total
4 | +
5 | print(calculate_total(12.5, 4))

F401 [*] `os` imported but unused
--> app.py:1:8

1 | import os
  | ^^
2 |
3 | from math_utils import calculate_total
  |
help: Remove unused import: `os`
- import os
1 |

Found 2 errors.
[*] 2 fixable with the `--fix` option.

```

图 1.14: `ruff check` 发现未使用 `import`

如图 1.15 所示，运行 `ruff check --fix app.py` 自动修复，输出显示 2 个错误全部被修复。

```

liuchenxi@liuchenxideMacBook-Air q06 % ruff check --fix app.py
Found 2 errors (2 fixed, 0 remaining).

```

图 1.15: `ruff check -fix` 修复问题

如图 1.16 所示，运行 `pytest`，输出显示 1 个测试通过，说明重命名后的代码仍然正确。

```

liuchenxi@liuchenxideMacBook-Air q06 % ruff check .
All checks passed!
liuchenxi@liuchenxideMacBook-Air q06 % pytest
===== test session starts =====
platform darwin -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0
rootdir: /Users/liuchenxi/Documents/系统开发工具/githubclone/lcx_course_work/w2/q06
collected 1 item

test_math_utils.py . [100%]

===== 1 passed in 0.00s =====

```

图 1.16: pytest 测试通过

## 1.2.2 实验结果

第六题的 github 的 commit 记录和 push 记录如图 1.17 所示。

```

liuchenxi@liuchenxideMacBook-Air w2 % git add q06
liuchenxi@liuchenxideMacBook-Air w2 % git commit -m "完成了q06"
[main f1e18bb] 完成了q06
5 files changed, 11 insertions(+)
create mode 100644 w2/q06/__pycache__/math_utils.cpython-314.pyc
create mode 100644 w2/q06/__pycache__/test_math_utils.cpython-314-pytest-9.1.1.pyc
create mode 100644 w2/q06/app.py
create mode 100644 w2/q06/math_utils.py
create mode 100644 w2/q06/test_math_utils.py
liuchenxi@liuchenxideMacBook-Air w2 % git push
Enter passphrase for key '/Users/liuchenxi/.ssh/id_ed25519':
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 10 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (10/10), 1.99 KiB | 1.99 MiB/s, done.
Total 10 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:lcx821/lcx_course_work.git
9f36f10..f1e18bb main -> main

```

图 1.17: github 记录 Q6

## 1.3 第 7 题用调试器定位归并排序缺陷

### 1.3.1 实验内容

如图 1.18 所示, 使用 vim 编辑器创建 merge\_sort.py, 内容为题目给定的存在缺陷的归并排序代码, 并用 cat 命令查看文件内容。其中 merge 函数 else 分支中的 result.append(right[i]) 处存在缺陷 (正确应为 right[j])。

```

liuchenxi@liuchenxideMacBook-Air q07 % vim merge_sort.py
liuchenxi@liuchenxideMacBook-Air q07 % cat merge_sort.py
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[i]); j += 1 # 此处存在缺陷
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))

```

图 1.18: 创建 merge\_sort.py

如图 1.19 所示, 直接运行 `merge_sort.py`, 可以看到输出结果错误 (或抛出异常), 确认代码存在缺陷。

```
liuchenxi@liuchenxideMacBook-Air q07 % python3 merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
```

图 1.19: 运行 `merge_sort.py` 结果错误

如图 1.20 所示, 使用 `python3 -m pdb merge_sort.py` 启动调试器: 先用 `l` 命令查看源码, 再在 `merge` 函数的第 8 行 (缺陷行) 使用 `b 8` 设置断点, 用 `c` 命令继续执行到断点处, 然后使用 `p` 命令打印 `i`、`j`、`left[i]`、`right[j]` 的值。从图中可以看到, 此时 `i=0`、`j=0`、`left[i]=3`、`right[j]=1`, 即 `left[i] > right[j]`, 本应把较小的 `right[j]` 加入结果, 但缺陷代码取的是 `right[i]`, 因此导致排序错误。

```
liuchenxi@liuchenxideMacBook-Air q07 % python3 -m pdb merge_sort.py
> /Users/liuchenxi/Documents/系统开发工具/githubclone/lcx_course_work/w2/q07/merge_sort.py(1)<module>()
-> def merge(left, right):
(Pdb) l
1  -> def merge(left, right):
2      result = []
3      i = j = 0
4      while i < len(left) and j < len(right):
5          if left[i] <= right[j]:
6              result.append(left[i]); i += 1
7          else:
8              result.append(right[i]); j += 1 # 此处存在缺陷
9      return result + left[i:] + right[j:]
10
11 def merge_sort(a):
(Pdb) b 8
Breakpoint 1 at /Users/liuchenxi/Documents/系统开发工具/githubclone/lcx_course_work/w2/q07/merge_sort.py:8
(Pdb) c
> /Users/liuchenxi/Documents/系统开发工具/githubclone/lcx_course_work/w2/q07/merge_sort.py(8)merge()
-> result.append(right[i]); j += 1 # 此处存在缺陷
(Pdb) p i, j, left[i], right[j]
(0, 0, 3, 1)
(Pdb) n
> /Users/liuchenxi/Documents/系统开发工具/githubclone/lcx_course_work/w2/q07/merge_sort.py(4)merge()
-> while i < len(left) and j < len(right):
(Pdb) q
```

图 1.20: 使用 `pdb` 调试并观察变量

在图 1.20 中, `b 8` 表示在第 8 行设置断点, `c` 表示继续执行, `p` 表示打印表达式, `n` 表示单步执行下一条语句, `q` 表示退出调试器。通过观察 `i`、`j`、`left[i]`、`right[j]` 的值, 可以定位到缺陷: `else` 分支中应使用 `right[j]` 而非 `right[i]`。将 `merge` 函数中的 `result.append(right[i])` 修改为 `result.append(right[j])` 即可完成最小修复, 没有用 `sorted` 替换归并排序。

如图 1.21 所示, 编写 `test_merge_sort.py`, 增加两个测试: 题目给定的输入 `[3, 1, 4, 1, 5, 9, 2, 6]` 和含重复元素的列表 `[5, 2, 2, 7, 1, 1, 5]`。

```
liuchenxi@liuchenxideMacBook-Air q07 % vim test_merge_sort.py
liuchenxi@liuchenxideMacBook-Air q07 % cat test_merge_sort.py
from merge_sort import merge_sort

def test_given_input():
    """测试题目给定的输入"""
    assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5, 6, 9]

def test_duplicate_elements():
    """测试含重复元素的列表"""
    assert merge_sort([5, 2, 2, 7, 1, 1, 5]) == [1, 1, 2, 2, 5, 5, 7]
```

图 1.21: 编写 `test_merge_sort.py` 测试

如图 1.22 所示, 运行 `pytest test_merge_sort.py -v`, 输出显示两个测试全部通过, 说明缺陷已被正确修复。

```

liuchenxi@liuchenxideMacBook-Air q07 % pytest test_merge_sort.py -v
===== test session starts =====
platform darwin -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- /opt/homebrew/Cellar/pytest/9.1.1/libexec/bin/python
cachedir: .pytest_cache
rootdir: /Users/liuchenxi/Documents/系统开发工具/githubclone/lcx_course_work/w2/q07
collected 2 items

test_merge_sort.py::test_given_input PASSED [ 50%]
test_merge_sort.py::test_duplicate_elements PASSED [100%]

===== 2 passed in 0.00s =====

```

图 1.22: 归并排序测试通过

### 1.3.2 实验结果

第七题的 github 的 commit 记录和 push 记录如图 1.23 所示。

```

liuchenxi@liuchenxideMacBook-Air w2 % git add q07
liuchenxi@liuchenxideMacBook-Air w2 % git commit -m "完成了 q07"
[main 2c74fcd] 完成了 q07
4 files changed, 28 insertions(+)
create mode 100644 w2/q07/__pycache__/merge_sort.cpython-314.pyc
create mode 100644 w2/q07/__pycache__/test_merge_sort.cpython-314-pytest-9.1.1.pyc
create mode 100644 w2/q07/merge_sort.py
create mode 100644 w2/q07/test_merge_sort.py
liuchenxi@liuchenxideMacBook-Air w2 % git push
[Enter passphrase for key '/Users/liuchenxi/.ssh/id_ed25519':
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 10 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 2.78 KiB | 2.78 MiB/s, done.
Total 9 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:lcx821/lcx_course_work.git
f1e18bb..2c74fcd main -> main

```

图 1.23: github 记录 Q7

## 1.4 第 8 题先测量，再优化慢速词频程序

### 1.4.1 实验内容

如图 1.24 所示，使用 vim 编辑器创建 generate\_words.py 和 wordfreq.py。generate\_words.py 使用固定的随机种子生成包含 1000 个不同词、共 30000 个词的 words.txt；wordfreq.py 通过列表 unique 逐词判断去重，统计不重复词的数量。

```

liuchenxi@liuchenxideMacBook-Air q08 % vim generate_words.py
liuchenxi@liuchenxideMacBook-Air q08 % cat generate_words.py
import random
random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))
liuchenxi@liuchenxideMacBook-Air q08 % vim wordfreq.py
liuchenxi@liuchenxideMacBook-Air q08 % cat wordfreq.py
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))

```

图 1.24: 创建 generate\_words.py 和 wordfreq.py

如图 1.25 所示，运行 python3 generate\_words.py 生成 words.txt。



```
liuchenxi@liuchenxideMacBook-Air q08 % python3 generate_words.py
```

图 1.25: 生成 words.txt

如图 1.26 所示, 使用 `time` 命令运行原程序 `wordfreq.py` 两次, 记录耗时。两次输出 `count=1000`, 耗时分别为 0.149s 和 0.127s。

```
liuchenxi@liuchenxideMacBook-Air q08 % time python3 wordfreq.py
count= 1000
python3 wordfreq.py  0.12s user 0.01s system 87% cpu 0.149 total
liuchenxi@liuchenxideMacBook-Air q08 % time python3 wordfreq.py
count= 1000
python3 wordfreq.py  0.11s user 0.01s system 97% cpu 0.127 total
```

图 1.26: 使用 `time` 运行原程序两次

如图 1.27 所示, 使用 `python3 -m cProfile -s cumulative wordfreq.py` 进行性能分析。从输出可以看到, 程序共 1012 次函数调用, 累计耗时 0.094s, 主要耗时集中在 `wordfreq.py` 第 1 行的模块级代码 (cumtime 0.094s), 即 `for` 循环中对列表 `unique` 的 `word not in unique` 判断。由于列表的成员判断复杂度为  $O(n)$ , 整体复杂度为  $O(n^2)$ , 这是程序的性能瓶颈。

```
liuchenxi@liuchenxideMacBook-Air q08 % python3 -m cProfile -s cumulative wordfreq.py
count= 1000
      1012 function calls in 0.094 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      1   0.094   0.094   0.094   0.094 {built-in method builtins.exec}
      1   0.093   0.093   0.094   0.094 wordfreq.py:1(<module>)
      1   0.000   0.000   0.000   0.000 {method 'split' of 'str' objects}
      1   0.000   0.000   0.000   0.000 {method 'read' of 'io.TextIOWrapper' objects}
    1000   0.000   0.000   0.000   0.000 {method 'append' of 'list' objects}
      1   0.000   0.000   0.000   0.000 {built-in method _io.open}
      1   0.000   0.000   0.000   0.000 <frozen codecs>:322(decode)
      1   0.000   0.000   0.000   0.000 {built-in method builtins.print}
      1   0.000   0.000   0.000   0.000 {built-in method _codecs.utf_8_decode}
      1   0.000   0.000   0.000   0.000 {method 'disable' of '_lsprof.Profiler' objects}
      1   0.000   0.000   0.000   0.000 <frozen codecs>:312(__init__)
      1   0.000   0.000   0.000   0.000 {built-in method builtins.len}
      1   0.000   0.000   0.000   0.000 <frozen codecs>:263(__init__)
```

图 1.27: 使用 `cProfile` 分析耗时

如图 1.28 所示, 创建优化后的 `wordfreq_opt.py`, 将去重用的列表 `unique` 改为集合 `set`: `unique = set(words)`。集合的成员判断平均复杂度为  $O(1)$ , 可以把整体复杂度从  $O(n^2)$  降到  $O(n)$ , 且不改变最终的 `count` 结果, 也没有写死 1000。

```
liuchenxi@liuchenxideMacBook-Air q08 % vim wordfreq_opt.py
liuchenxi@liuchenxideMacBook-Air q08 % cat wordfreq_opt.py
words = open("words.txt", encoding="utf-8").read().split()
unique = set(words)
print("count=", len(unique))
```

图 1.28: 创建优化后的 `wordfreq_opt.py`

如图 1.29 所示, 使用相同输入再次运行优化后的程序两次, 耗时分别为 0.053s 和 0.043s。

```
[liuchenxi@liuchenxideMacBook-Air q08 % time python3 wordfreq_opt.py
count= 1000
python3 wordfreq_opt.py 0.02s user 0.01s system 55% cpu 0.053 total
[liuchenxi@liuchenxideMacBook-Air q08 % time python3 wordfreq_opt.py
count= 1000
python3 wordfreq_opt.py 0.02s user 0.01s system 74% cpu 0.043 total
```

图 1.29: 使用 `time` 运行优化程序两次

优化前两次耗时为 0.149s 和 0.127s，中位数为 0.138s；优化后两次耗时为 0.053s 和 0.043s，中位数为 0.048s。加速比为  $0.138/0.048 \approx 2.9$ ，即优化后性能约提升到原来的 2.9 倍，且最终 count 仍为 1000，结果保持不变。

### 1.4.2 实验结果

第八题的 github 的 commit 记录和 push 记录如图 1.30 所示。

```
[liuchenxi@liuchenxideMacBook-Air w2 % git add q08
[liuchenxi@liuchenxideMacBook-Air w2 % git commit -m "完成了 q08"
[main c923246] 完成了 q08
4 files changed, 15 insertions(+)
create mode 100644 w2/q08/generate_words.py
create mode 100644 w2/q08/wordfreq.py
create mode 100644 w2/q08/wordfreq_opt.py
create mode 100644 w2/q08/words.txt
[liuchenxi@liuchenxideMacBook-Air w2 % git push
[Enter passphrase for key '/Users/liuchenxi/.ssh/id_ed25519':
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 10 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (8/8), 53.05 KiB | 2.53 MiB/s, done.
Total 8 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To github.com:lcx821/lcx_course_work.git
2c74fcd..c923246 main -> main
```

图 1.30: github 记录 Q8

## 1.5 Github 仓库地址

本次课程的报告、练习、代码等，均已上传至个人 github 仓库。  
Github 仓库地址：[https://github.com/lcx821/lcx\\_course\\_work](https://github.com/lcx821/lcx_course_work)

## 课后练习内容和结果

### 2.1 基础命令与重定向

#### 题 1：确认 Shell 环境

**要求：**本课程要求使用类 Unix Shell (Bash 或 ZSH)。运行 `echo $SHELL` 确认。

**解答：**

```
echo $SHELL
```

**实际输出：**

```
/bin/zsh
```

**说明：**当前默认 Shell 是 zsh，属于类 Unix Shell，符合课程要求。bash 也可用（版本 3.2.57）。Windows 用户需使用 WSL 或 Linux 虚拟机，不能用 `cmd.exe` 或 PowerShell。

#### 题 2：ls -l 的输出格式

**要求：**ls -l 的 -l 选项作用是什么？运行 `ls -l /`，每一行最前面的 10 个字符分别代表什么？

**解答：**

```
ls -l /
```

**实际输出（节选）：**

```
total 10
drwxrwxr-x 42 root  admin 1344 Aug 30 13:55 Applications
drwxr-xr-x@ 39 root  wheel 1248 Apr  6 16:10 bin
lrwxr-xr-x@  1 root  wheel  11 Apr  6 16:10 etc -> private/etc
```

**-l 选项作用：**以长格式 (long format) 列出文件，显示文件类型、权限、硬链接数、所有者、所属组、大小、修改时间和文件名。

**前 10 个字符的含义：**

例如 `drwxr-xr-x` 表示：这是一个目录，所有者可读写执行，组用户可读执行，其他用户可读执行。

| 位置       | 含义               | 示例                         |
|----------|------------------|----------------------------|
| 第 1 位    | 文件类型             | d = 目录, l = 符号链接, - = 普通文件 |
| 第 2-4 位  | 所有者 (user) 权限    | rwX = 读、写、执行               |
| 第 5-7 位  | 所属组 (group) 权限   | r-X = 读、执行 (无写权限)          |
| 第 8-10 位 | 其他用户 (others) 权限 | r-X = 读、执行                 |

注：第 11 位可能是 @ (macOS 扩展属性) 或 + (ACL)，不属于权限位。

### 题 3：glob 模式匹配

要求：什么是 glob？测试 `ls *.txt`、`ls file?.txt`、`ls \{a,b,c\}.txt` 等模式。

glob 是什么：glob 是 Shell 提供的文件名通配符模式匹配机制。Shell 在执行命令前，会先将包含通配符的参数展开为匹配的文件名列表，再传给程序。

| 模式        | 含义                   |
|-----------|----------------------|
| *         | 匹配任意长度的任意字符（包括空）     |
| ?         | 匹配恰好一个任意字符           |
| [abc]     | 匹配 a、b、c 中任意一个字符     |
| [a-z]     | 匹配 a 到 z 范围内任意一个字符   |
| \{a,b,c\} | 大括号扩展，生成 a、b、c 多个字符串 |

#### 实际测试：

```
mkdir glob_test && cd glob_test
touch file1.txt file2.txt file3.txt a.txt b.txt c.txt abc.txt ab.txt

ls *.txt           # 匹配所有 .txt 文件
ls file?.txt       # 匹配 file + 任意一个字符 + .txt
ls {a,b,c}.txt     # 大括号扩展为 a.txt b.txt c.txt
ls [ab]*.txt       # 匹配以 a 或 b 开头的 .txt 文件
```

#### 输出：

```
=== ls *.txt ===
a.txt ab.txt abc.txt b.txt c.txt file1.txt file2.txt file3.txt

=== ls file?.txt ===
file1.txt file2.txt file3.txt

=== ls {a,b,c}.txt ===
a.txt b.txt c.txt

=== ls [ab]*.txt ===
a.txt ab.txt abc.txt b.txt
```



题 4：三种引号的区别

要求：' 单引号'、" 双引号" 和 \\$\$'ANSI 引号' 有什么区别？写一条命令，输出同时包含字面量 \、! 和换行符的字符串。

| 引号类型            | 变量展开 | 转义序列   | 历史扩展 | 适用场景       |
|-----------------|------|--------|------|------------|
| '...' 单引号       | 否    | 否      | 否    | 纯字面量字符串    |
| "..." 双引号       | 是    | 否（除少数） | 可能触发 | 需要嵌入变量     |
| \\$'...' ANSI-C | 否    | 是      | 否    | 需要转义序列的字面量 |

实际测试：

```
# 单引号：完全字面量
echo 'It costs $100! Hello\nWorld'
# 输出：It costs $100! Hello\nWorld

# 双引号：展开变量，\n 不解释
price=100
echo "It costs $price! Hello\nWorld"
# 输出：It costs $price! Hello\nWorld

# ANSI-C 引号：解释 \n, $ 是字面量
echo '$It costs $100! Hello\nWorld'
# 输出：
# It costs $100! Hello
# World
```

题目要求的命令（同时包含字面量 \、! 和换行符）：

```
echo '$价格是 $100!\n这是第二行'
```

输出：

```
价格是 $100!
这是第二行
```

也可以用 `printf '价格是 \n这是第二行'`，`printf` 默认就解释转义序列。

题 5：标准流与重定向

要求：Shell 有三条标准流：stdin(0)、stdout(1)、stderr(2)。运行 `ls /nonexistent /tmp`，把 stdout 和 stderr 分别重定向到两个文件。如何把两者都重定向到同一个文件？

三条标准流：

- **stdin (0)**：标准输入，默认来自键盘
- **stdout (1)**：标准输出，默认打印到终端
- **stderr (2)**：标准错误，默认也打印到终端（独立于 stdout）

分别重定向到两个文件：

```
ls /nonexistent /tmp > stdout.txt 2> stderr.txt
```

- `> stdout.txt` 等价于 `1> stdout.txt`，把标准输出写入 `stdout.txt`
- `2> stderr.txt` 把标准错误写入 `stderr.txt`

实际输出：

```
stdout.txt: 包含 /tmp 目录下的文件列表
stderr.txt: 包含 "ls: /nonexistent: No such file or directory"
```

两者重定向到同一个文件（两种方法）：

方法 1（POSIX 标准，推荐）：

```
ls /nonexistent /tmp > both.txt 2>&1
```

- `> both.txt` 先把 `stdout` 指向 `both.txt`
- `2>&1` 把 `stderr` 重定向到当前 `stdout` 指向的地方（即 `both.txt`）
- 顺序很重要：`2>&1` 必须在 `>` 之后

方法 2（bash/zsh 简写）：

```
ls /nonexistent /tmp &> both.txt
```

## 题 6：退出状态与 `&&` / `||`

要求：`\$?` 保存上一条命令的退出状态（0= 成功）。`&&` 仅在前一条成功时执行后一条；`||` 仅在前一条失败时执行后一条。写一行命令：仅当 `/tmp/mydir` 不存在时才创建它。

退出状态：每个命令执行后返回一个 0-255 的整数，0 表示成功，非 0 表示失败。可用 `echo \$?` 查看。

逻辑运算符：

- `cmd1 && cmd2`：cmd1 成功（退出码 0）才执行 cmd2
- `cmd1 || cmd2`：cmd1 失败（退出码非 0）才执行 cmd2
- 两者可组合：`cmd1 && cmd2 || cmd3`（类似三元运算符）

题目要求的命令：

```
[ -d /tmp/mydir ] || mkdir /tmp/mydir
```

解释：

- `[ -d /tmp/mydir ]` 测试目录是否存在（`test -d` 的简写）
- 如果存在，[ 命令返回 0（成功），|| 后面的 `mkdir` 不执行
- 如果不存在，[ 返回非 0（失败），|| 触发 `mkdir /tmp/mydir`

实际验证：

```
# 第一次运行：目录不存在，创建成功
[ -d /tmp/mydir ] || mkdir /tmp/mydir
ls -ld /tmp/mydir
# 输出: drwxr-xr-x  2 liuchenxi  wheel  64 Aug 30 16:51 /tmp/mydir

# 第二次运行：目录已存在，不重复创建
[ -d /tmp/mydir ] || mkdir /tmp/mydir
# 无输出，无报错
```

更简洁的等价写法: `mkdir -p /tmp/mydir` (`-p` 表示目录已存在时不报错), 但题目要求用 `\&\&///` 逻辑。

## 2.2 脚本编写与权限

### 题 7: 为什么 `cd` 必须是 Shell 内建命令?

**要求:** 为什么 `cd` 必须是 Shell 内建命令, 而不能是独立程序? (提示: 想想子进程能影响和不能影响父进程的哪些状态。)

**核心原因:** 子进程无法修改父进程的工作目录。

**详细解释:**

1. **进程独立性:** 在 Unix 系统中, 每个进程都有自己独立的「当前工作目录」(`cwd`)。当 Shell 执行一个外部程序时, 会通过 `fork()` 创建子进程, 再用 `exec()` 加载程序。子进程继承父进程的 `cwd`, 但这只是一个副本。
2. **如果 `cd` 是独立程序:**
  - Shell `fork` 出子进程
  - 子进程执行 `cd /some/path`, 修改的是子进程自己的 `cwd`
  - 子进程结束后, Shell (父进程) 的 `cwd` 完全没有改变
  - 结果就是: 你执行了 `cd /some/path`, 但提示符显示的当前目录没变
3. **内建命令的优势:** 内建命令由 Shell 自身执行, 不创建子进程, 可以直接修改 Shell 进程自己的 `cwd`, 从而真正改变后续命令的工作目录。

**验证:**

```
which cd
# 输出: cd: shell built-in command (或 no cd in $PATH)
# 说明 cd 不是独立程序, 而是 Shell 内建命令
```

**同类内建命令:** `cd`、`export`、`alias`、`exit`、`source/.` 等——凡是需要修改 Shell 自身状态的命令, 都必须是内建命令。

### 题 8: 写脚本检查文件是否存在

**要求:** 写一个脚本, 接收文件名参数 (`\$1`), 用 `test -f` 或 `[ -f ... ]` 检查该文件是否存在, 并根据结果输出不同提示。

**脚本 `check.sh`:**

```
#!/bin/bash
# 检查文件是否存在的脚本

if [ -f "$1" ]; then
    echo "文件 '$1' 存在。"
else
    echo "文件 '$1' 不存在。"
fi
```

**关键点：**

- `\$1` 是第一个命令行参数
- `[ -f "$1" ]` 测试 `\$1` 是否为存在的普通文件 (`-f = file`)
- `"\$1"` 加双引号防止文件名含空格时出错
- `if ...; then ... else ... fi` 是 `bash` 条件语句结构

**其他常用测试条件：**

| 条件                   | 含义        |
|----------------------|-----------|
| <code>-f file</code> | 存在且是普通文件  |
| <code>-d file</code> | 存在且是目录    |
| <code>-e file</code> | 存在（任何类型）  |
| <code>-r file</code> | 存在且可读     |
| <code>-w file</code> | 存在且可写     |
| <code>-x file</code> | 存在且可执行    |
| <code>-s file</code> | 存在且大小大于 0 |

**题 9：chmod +x 的作用**

**要求：**把上一题的脚本保存为 `check.sh`。先运行 `./check.sh somefile`，会发生什么？然后执行 `chmod +x check.sh` 再试一次。为什么这一步是必须的？

**chmod 前运行：**

```
./check.sh somefile
# 输出: bash: ./check.sh: Permission denied
```

**chmod 前的权限：**

```
-rw----- 1 liuchenxi staff 147 Aug 30 16:52 check.sh
```

`rw-` = 所有者可读、可写，但不可执行

**执行 chmod +x check.sh 后：**

```
-rwx--x--x 1 liuchenxi staff 147 Aug 30 16:52 check.sh
```

`rwx` = 所有者可读、可写、可执行

**chmod 后运行：**

```
./check.sh somefile
# 输出: 文件 'somefile' 不存在。

./check.sh check.sh
# 输出: 文件 'check.sh' 存在。
```

**为什么必须 chmod +x：**

- Unix 系统中，文件能否被直接执行取决于可执行权限位（`x` 位），而不是文件扩展名
- 新建文件默认没有执行权限（安全考虑）
- `chmod +x` 给所有用户添加执行权限，Shell 才能通过 `./check.sh` 直接运行它
- 没有执行权限时，仍可通过解释器显式运行：`bash check.sh somefile`

shebang 行（`#!/bin/bash`）只在文件有执行权限且被直接运行时才生效。

题 10: `set -x` 的作用

要求：在脚本的 `set` 选项里加入 `-x` 会发生什么？写个简单脚本试试。

`set -x` 的作用：开启调试模式（`xtrace`）。Shell 在执行每条命令前，会先把该命令及其展开后的参数打印到 `stderr`（通常以 `+` 开头），方便追踪脚本执行流程。

演示脚本 `debug\_demo.sh`：

```
#!/bin/bash
set -x

echo "开始执行脚本"
name="Shell"
echo "你好, $name"
for i in 1 2 3; do
    echo "第 $i 次循环"
done
echo "脚本结束"
```

运行输出（`+` 开头的行是 `set -x` 打印的追踪信息）：

```
+ echo '开始执行脚本'
开始执行脚本
+ name=Shell
+ echo '你好, Shell'
你好, Shell
+ i=1
+ echo '第 1 次循环'
第 1 次循环
+ i=2
+ echo '第 2 次循环'
第 2 次循环
+ i=3
+ echo '第 3 次循环'
第 3 次循环
+ echo '脚本结束'
脚本结束
```

常用 `set` 选项总结：

| 选项                           | 作用                   |
|------------------------------|----------------------|
| <code>set -e</code>          | 任何命令失败时立即退出脚本        |
| <code>set -u</code>          | 使用未定义变量时报错（而非当作空字符串） |
| <code>set -o pipefail</code> | 管道中任一命令失败，整个管道视为失败   |
| <code>set -x</code>          | 打印每条执行的命令（调试用）       |

推荐组合：`set -euo pipefail`（脚本开头的「严格模式」），调试时再加 `-x`。

## 题 11：带日期的备份文件

**要求：**写一条命令，把文件复制为带当天日期的备份文件名（例如 `notes.txt` → `notes\_2026-01-12.txt`）。提示： `\$(date +%Y-%m-%d)`

**命令：**

```
cp notes.txt "notes_$(date +%Y-%m-%d).txt"
```

**原理：**

- `\$(date +%Y-%m-%d)` 是**命令替换**：执行 `date` 命令，用其输出替换整个 `\$( )`
- `+%Y-%m-%d` 是 `date` 的格式参数：`%Y` = 四位年，`%m` = 两位月，`%d` = 两位日
- 双引号确保日期展开后作为一个整体参数

**实际运行：**

```
echo "这是笔记内容" > notes.txt
cp notes.txt "notes_$(date +%Y-%m-%d).txt"
ls -l notes*.txt
```

**输出：**

```
-rw-r--r--  1 staff  19 Aug 30 16:52 notes.txt
-rw-r--r--  1 staff  19 Aug 30 16:52 notes_2026-08-30.txt
```

**通用公式：**

```
cp "文件名" "文件名_$(date +%Y-%m-%d).扩展名"
```

**其他常用日期格式：**

```
date +%Y%m%d           # 20260830
date +%Y-%m-%d_%H%M%S  # 2026-08-30_165200（带时间）
date -v-1d +%Y-%m-%d   # 昨天的日期（macOS）
```

## 题 12：修改 flaky test 脚本接收命令行参数

**要求：**修改讲义中的「复现偶尔才会失败的测试」脚本，使它能够从命令行参数接收测试命令，而不是写死 `cargo test my\_test`。提示：`\$1` 或 `\$0`

**修改后的脚本 `flaky\_test.sh`：**

```
#!/bin/bash
set -euo pipefail

# 从命令行参数接收测试命令，而不是写死
TEST_CMD="$@"

if [ -z "$TEST_CMD" ]; then
    echo "用法：$0 <测试命令>"
    echo "示例：$0 cargo test my_test"
    exit 1
fi
```



```
# Start CPU stress in background
stress --cpu 8 &
STRESS_PID=$!

# Setup log file
LOGFILE="test_runs_$(date +%s).log"
echo "Logging to $LOGFILE"
echo "运行测试命令: $TEST_CMD"

# Run tests until one fails
RUN=1
while $TEST_CMD > "$LOGFILE" 2>&1; do
    echo "Run $RUN passed"
    ((RUN++))
done

# Cleanup and report
kill $STRESS_PID
echo "Test failed on run $RUN"
echo "Last 20 lines of output:"
tail -n 20 "$LOGFILE"
echo "Full log: $LOGFILE"
```

关键改动:

- 原脚本写死: while cargo test my\\_test > \"\$LOGFILE\" 2>&1; do
- 改为: TEST\\_CMD=\"\$@\" 接收所有命令行参数, 然后 while \"\$TEST\\_CMD\" > ...

特殊参数说明:

| 参数    | 含义                              |
|-------|---------------------------------|
| \\$1  | 第一个参数                           |
| \\$@  | 所有参数, 每个参数独立 (加引号后\"\$@\" 保留空格) |
| \\$*  | 所有参数, 用 IFS 连接成一个字符串            |
| \\$## | 参数个数                            |
| \\$0  | 脚本自身名称                          |
| \\$!  | 最近一个后台进程的 PID                   |

使用方式:

```
./flaky_test.sh cargo test my_test      # 传 cargo test 命令
./flaky_test.sh pytest tests/           # 传 pytest 命令
./flaky_test.sh false                   # 用 false 测试 (第一次就失败)
```

注意: \"\$@\" 比 \"\$1\" 更适合接收完整命令, 因为命令可能包含多个参数 (如 cargo test my\\_test 是三个参数)。

## 2.3 管道、工具与进阶

### 题 13: 找出 home 目录最常见的 5 种文件扩展名

要求: 使用管道找出 home 目录中最常见的 5 种文件扩展名。提示: 组合 `find`、`grep`/`sed`/`awk`、`sort`、`uniq -c`、`head`

命令:

```
find ~ -type f 2>/dev/null \
| grep -oE '\.[^./]+$' \
| sort \
| uniq -c \
| sort -rn \
| head -n 5
```

逐步拆解:

1. `find ~ -type f`: 递归列出 home 目录下所有普通文件
2. `2>/dev/null`: 把权限错误等 `stderr` 丢弃
3. `grep -oE '\textbackslash.[^{}./]+\textbackslash$'`: 提取每行末尾的扩展名
  - `-o`: 只输出匹配部分
  - `-E`: 使用扩展正则
  - `\textbackslash.[^{}./]+\textbackslash$`: 匹配. + 一个或多个非. 非/ 字符 + 行尾
4. `sort`: 排序 (相同扩展名排在一起)
5. `uniq -c`: 去重并统计每行出现次数
6. `sort -rn`: 按数字逆序排序 (出现次数多的在前)
7. `head -n 5`: 取前 5 个

实际输出:

```
11425 .py
9438 .js
6911 .pyc
5855 .md
3948 .json
```

等价的 `awk` 版本:

```
find ~ -type f 2>/dev/null | awk -F. '{print ".$NF"}' | sort | uniq -c | sort -rn |
head -5
```

### 题 14: `xargs` 结合 `find` 统计 `.sh` 文件行数

要求: `xargs` 把 `stdin` 的每一行转换为命令参数。结合 `find` 和 `xargs` (不要用 `find -exec`), 找出目录中所有 `.sh` 文件, 并用 `wc -l` 统计每个文件行数。加分项: 正确处理文件名中的空格。提示: `-print0` 和 `-0`

基础版本:

```
find . -name "*.sh" | xargs wc -l
```

**问题：**如果文件名包含空格（如 `my script.sh`），`xargs` 会把它拆成两个参数 `my` 和 `script.sh`，导致错误。

**正确处理空格的版本（加分项）：**

```
find . -name "*.sh" -print0 | xargs -0 wc -l
```

**原理：**

- `find -print0`：用 **null 字符（\0）** 分隔文件名，而不是换行符
- `xargs -0`：以 **null 字符** 作为分隔符读取输入
- 因为文件名中不可能包含 **null 字符**，所以这种方式 **100% 安全** 处理任何文件名（含空格、换行、特殊字符）

**实际输出：**

```
1 ./dir with spaces/my script.sh
34 ./flaky_test.sh
8 ./check.sh
10 ./debug_demo.sh
53 total
```

**xargs 常用参数：**

| 参数                   | 含义                                           |
|----------------------|----------------------------------------------|
| <code>-0</code>      | 以 null 字符分隔输入（配合 <code>find -print0</code> ） |
| <code>-n N</code>    | 每次传递 N 个参数给命令                                |
| <code>-I \{\}</code> | 用 <code>\{\}</code> 作为参数占位符                  |
| <code>-P N</code>    | 并行运行 N 个进程                                   |

## 题 15: curl 获取课程网站 HTML，统计列出了多少讲

**要求：**使用 `curl` 获取课程网站的 HTML，通过 `grep` 统计列出了多少讲。提示：找出每讲课程名称在 HTML 中的共性；用 `curl -s` 关闭进度输出。

**命令：**

```
curl -s https://missing-semester-cn.github.io/2026/ \
| grep -oE 'href="/2026/[a-z-]+/"' \
| sort -u \
| wc -l
```

**逐步拆解：**

1. `curl -s URL`：静默模式获取网页 HTML（不显示进度条）
2. `grep -oE 'href="/2026/[a-z-]+/'`：提取所有课程链接
  - 课程链接的共性：`href="/2026/课程名/"`
  - 课程名由小写字母和连字符组成
3. `sort -u`：排序并去重（`-u = unique`）
4. `wc -l`：统计行数，即课程讲数

实际输出：

```
9
```

列出各讲名称：

```
curl -s https://missing-semester-cn.github.io/2026/ \
| grep -oE 'href="/2026/[a-z-]+/"' \
| sort -u \
| sed 's|href="/2026/||;s|/||'
```

输出：

```
agentic-coding
beyond-code
code-quality
command-line-environment
course-shell
debugging-profiling
development-environment
shipping-code
version-control
```

共 9 讲，与课程介绍中「九场 1 小时的讲座」一致。

## 题 16: jq 处理 JSON

要求：用 curl 获取示例数据，再用 jq 提取 version 大于 6 的人员姓名。提示：先 jq . 看结构；再试 jq '.[0] | select(...)' | .name'

获取数据并查看结构：

```
curl -s https://microsoftedge.github.io/Demos/json-dummy-data/64KB.json > data.json
jq '.[0]' data.json
```

JSON 结构：

```
{
  "name": "Adeel Solangi",
  "language": "Sindhi",
  "id": "V590F92YF627HFO",
  "bio": "Donec lobortis eleifend...",
  "version": 6.1
}
```

顶层是一个数组，每个元素包含 name、language、id、bio、version 字段。

提取 version > 6 的人员姓名：

```
jq '.[0] | select(.version > 6) | .name' data.json
```

jq 语法拆解：

- .[]：遍历数组中的每个元素
- select(.version > 6)：筛选 version 字段大于 6 的元素
- .name：提取 name 字段

统计数量：

```
jq '[.[] | select(.version > 6)] | length' data.json
# 输出：89
```

部分输出：

```
"Adeel Solangi"
"Aamir Solangi"
"Adil Eli"
"George Mifsud"
"John Falzon"
...
```

jq 常用操作速查：

| 表达式          | 含义             |
|--------------|----------------|
| .            | 整个输入           |
| .field       | 取字段            |
| .[]          | 遍历数组           |
| select(cond) | 筛选满足条件的元素      |
| length       | 数组长度或字符串长度     |
| keys         | 对象的所有键         |
| map(expr)    | 对数组每个元素应用 expr |

题 17：awk 按列过滤并交换列

要求：写一个 awk 命令：只输出第二列大于 100 的行，并交换第一列和第三列。测试数据：printf 'a 50 x\textrightarrow nb 150 y\textrightarrow nc 200 z\textrightarrow n'

命令：

```
printf 'a 50 x\rightarrow nb 150 y\rightarrow nc 200 z\rightarrow n' \
| awk '$2 > 100 { temp=$1; $1=$3; $3=temp; print }'
```

awk 语法拆解：

- \ \$2 > 100 : 模式（条件），只处理第二列大于 100 的行
- \ { temp=\ \$1; \ \$1=\ \$3; \ \$3=temp; print \ } : 动作
  - temp=\ \$1 : 把第一列存到临时变量
  - \ \$1=\ \$3 : 第一列替换为第三列的值
  - \ \$3=temp : 第三列替换为原来的第一列
  - print : 打印修改后的整行

实际输出：

```
y 150 b
z 200 c
```

验证：

- 原始行 a 50 x：第二列 50，不大于 100，被过滤
- 原始行 b 150 y：第二列 150 > 100，交换第一列 (b) 和第三列 (y) → y 150 b
- 原始行 c 200 z：第二列 200 > 100，交换第一列 (c) 和第三列 (z) → z 200 c

awk 核心概念：

- awk 按行处理，每行自动按空白字符分成列
- \$1, \$2, \$3... 引用各列，\$0 表示整行
- NR = 当前行号，NF = 当前行列数
- FS = 输入分隔符（默认空白），-F, 可设为逗号
- BEGIN {} 在处理前执行，END {} 在处理后执行

题 18：拆解 SSH 日志管道 + 从历史记录找出最常用命令

要求：拆解讲义中的 SSH 日志处理管道：每一步分别做了什么？然后仿照它构建一个管道，从 ~/.bash\_history（或 ~/.zsh\_history）中找出最常使用的 Shell 命令。

第一部分：拆解 SSH 日志管道

原始管道：

```
ssh myserver 'journalctl -u sshd -b-1 | grep "Disconnected from" \
| sed -E 's/.*Disconnected from .* user (.*?) [^ ]+ port.*/\1/' \
| sort | uniq -c \
| sort -nk1,1 | tail -n10 \
| awk '{print $2}' | paste -sd,
```

逐步拆解：

| 步骤 | 命令                               | 作用                                    |
|----|----------------------------------|---------------------------------------|
| 1  | ssh myserver '...'               | 登录远程服务器，在远程执行引号内的命令                   |
| 2  | journalctl -u sshd -b-1          | 查看 sshd 服务在上一次启动时的系统日志                |
| 3  | grep "Disconnected from"         | 筛选包含「断开连接」的日志行                        |
| 4  | sed -E 's/...user (.*?) .../\1/' | 用正则提取用户名，(.*?) 捕获，\textbackslash 1 输出 |
| 5  | sort                             | 对用户名排序（相同名字排在一起）                      |
| 6  | uniq -c                          | 去重并统计每个用户名出现次数                        |
| 7  | sort -nk1,1                      | 按第一列（次数）数值排序（升序）                      |
| 8  | tail -n10                        | 取最后 10 个，即出现次数最多的 10 个                |
| 9  | awk '{print \$2}'                | 只输出第二列（用户名），去掉次数                      |
| 10 | paste -sd,                       | 把多行合并为一行，用逗号分隔                        |

最终输出示例：

```
postgres,mysql,oracle,dell,ubuntu,inspur,test,admin,user,root
```

## 第二部分：从历史记录找出最常用命令

命令 (zsh 用户):

```
LC_ALL=C awk '{print $1}' ~/.zsh_history | sort | uniq -c | sort -rn | head -n 10
```

命令 (bash 用户):

```
awk '{print $1}' ~/.bash_history | sort | uniq -c | sort -rn | head -n 10
```

逐步拆解:

1. `awk '\{print \$1\}'`: 提取每行的第一个词 (即命令名, 去掉参数)
2. `sort`: 排序
3. `uniq -c`: 统计每个命令出现次数
4. `sort -rn`: 按次数逆序排序
5. `head -n 10`: 取前 10 个

实际输出 (zsh\_history):

```
123 brew
25 cd
22 git
20 sudo
14 ls
9 echo
8 fc-match
7 which
7 find
6 mkdir
```

说明:

- zsh 的历史文件默认格式可能包含时间戳前缀 ( : 1234567890:0;command ), 如果有, 需要先用 `sed 's/\~{}: [0-9]*:[0-9]*;/'` 去掉前缀
- `LC_ALL=C` 防止遇到非 UTF-8 字节时 `awk/sed` 报错
- 最常用的命令是 `brew` (macOS 包管理器), 其次是 `cd`、`git`、`sudo`、`ls`



## 解题感悟

太难了!!! 对于这一门课程,我是 0 基础,不少同学也都是零基础。即使我认真看完 *The Missing Semester of Your CS Education* 的讲座视频讲解,并读完讲义后,还是难以完成课堂实验。比如第一步,在终端用命令行创建文件夹,课程中根本没有涉及,我通过大模型的帮助,才知道需要用 `mkdir` 这一个命令来创建文件夹。还有不少 `shell` 命令,单纯我自己是没办法写出来的。不少正则表达式如同课程讲解人 Jon 说的那样,短时间内很难掌握。然后完成任务的过程中,我有时还忘记了截图,比如完成第三题时,我就忘记了截图。写实验报告时,我发现我根本没有第三题的截图,我一度想要通过 Github 上的 `commit` 记录替代终端上的截图,但后面发现这根本没办法满足题目的要求,于是我又重新做了一遍第三题,才将第三题所需要的终端执行窗口截图补齐。

还有 `latex` 的编写,我之前从未接触过 `latex` 的编写,只是听说过 `latex` 对于写论文很有用。老师要求自己设计模板,我觉得这对目前的我来说,在这一周的时间里面很难做到。于是我通过网络搜索到了于景一学长放在 `github` 上的中国海洋大学实验报告 `latex` 模板,用他的模板完成了实验报告的编写。最初,我准备在 `overleaf` 上编写实验报告,因为在线编写可以省去本地配置环境的麻烦。但是后面我看到要求将报告同步至 Github 仓库,于是我又重新在本地部署了一遍 `latex`。期间遇到不少麻烦,比如有三款字体无法识别。其中两款字体没有在我的电脑上安装,有一款字体没有被正确识别。`vs code` 工作区一度出现了两千多条报错,后面单是为了解决字体的问题,我就耗费了不少的时间。

总体来说,这次的实验无疑是非常困难的,我用了两三天才勉强完成了实验和课堂报告的撰写。困难主要是对 `shell` 和 `latex` 完全不熟悉,但是为 `latex` 配置环境就用了不少时间。但收获还是有的,通过 Jon 的 Youtube 视频,我对 `shell` 命令有了一个了解,第一次知道了 `shell` 的强大。我发现这门课程学习的内容都挺有意思的,无论是对 `shell` 的学习,`latex` 的运用以及 Github 的使用,都是我们未来会接触到的。具有不可替代的现实意义。这是我学习到的内容最实用的一次。通过这次实验,我也基本了解了 `shell` 的使用,并真正直观感受到了 Github 应该怎么用,以及为什么不少人都说 Github 很好用。希望后面的课程我能更加顺利!!!

